MODULE 2: SQL DATABASE AND QUERIES

Time: 1-2 hours

Business Requirements

The sales team wants to query the sales data efficiently. Your task is to:

1. Set up a PostgreSQL database with the sales data
2. Write queries to answer key business questions:
    - What is our total revenue?
    - Which products are top performers?
    - How do different regions compare?
    - What are the monthly trends?
3. Create a reusable view for common analysis
4. Document your queries so others can use them

Detailed Task Breakdown

Part 1: Set Up PostgreSQL Database

Step 1: Create Database

Using DBeaver, create a new database named "sales_db".

Step 2: Create the Sales Table

Write and execute a CREATE TABLE statement with appropriate columns and data types.

Step 3: Import Data

Import sales_data.csv into your sales table.

Step 4: Verify Import

Run a query to check the row count.

Part 2: Write Analysis Queries

Step 5: Overall Sales Summary

Write a query to calculate total revenue, transaction count, and average sale.

Step 6: Product Performance

Write a query showing revenue by product, sorted by highest revenue first.

Step 7: Regional Analysis

Write a query comparing sales across different regions.

Step 8: Monthly Trends

Write a query showing revenue by month throughout the year.

Step 9: Top Products by Volume

Write a query to find the top 5 products by quantity sold.

Part 3: Create a Reusable View

Step 10: Create Product Performance View

Create a view called "product_performance" that shows key metrics for each product including revenue, units sold, and percentage of total revenue.

Step 11: Test the View

Query your view to make sure it works correctly.

Part 4: Document Your Work

Step 12: Add Documentation

Add a documentation header to your SQL file explaining the purpose, database structure, and available queries.

Deliverable

Save your work as **sales_queries.sql**

Your file should contain:

- Documentation header
- CREATE TABLE statement
- 5 analysis queries with comments explaining their purpose
- 1 CREATE VIEW statement
- All queries tested and working

Need Help?

Part 1: Set Up PostgreSQL Database

Step 1: Create Database

1. Open DBeaver
2. Right-click on your PostgreSQL connection in the Database Navigator
3. Select "Create New Database"
4. Name it "sales_db"
5. Click OK

Step 2: Create the Sales Table

Open a new SQL script and run this:

```sql
CREATE TABLE sales (
    sale_id INTEGER PRIMARY KEY,
    date DATE NOT NULL,
    product VARCHAR(50) NOT NULL,
    quantity INTEGER NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    region VARCHAR(20)
);
```

Step 3: Import Data

1. Right-click on the "sales" table in Database Navigator
2. Select "Import Data"
3. Choose your sales_data.csv file
4. Map the columns (should auto-map correctly)
5. Click "Start"

Step 4: Verify Import

Run this query:

```sql
SELECT COUNT(*) FROM sales;
```

You should see approximately 1,178 rows.

Part 2: Write Analysis Queries

Step 5: Overall Sales Summary

```sql
-- Query 1: Total revenue and sales count
SELECT
   COUNT(*) as total_transactions,
   SUM(total) as total_revenue,
   AVG(total) as average_sale
FROM sales;
```

Step 6: Product Performance

```sql
```

```sql
-- Query 2: Revenue by product
SELECT
    product,
    COUNT(*) as num_sales,
    SUM(quantity) as units_sold,
    SUM(total) as revenue
FROM sales
GROUP BY product
ORDER BY revenue DESC;
```

Step 7: Regional Analysis

```sql
-- Query 3: Sales by region
SELECT
    region,
    COUNT(*) as num_sales,
    SUM(total) as revenue,
    ROUND(AVG(total), 2) as avg_sale
FROM sales
GROUP BY region
ORDER BY revenue DESC;
```

Step 8: Monthly Trends

```sql
-- Query 4: Monthly sales trend
SELECT
    TO_CHAR(date, 'YYYY-MM') as month,
    COUNT(*) as num_sales,
    SUM(total) as monthly_revenue
FROM sales
GROUP BY TO_CHAR(date, 'YYYY-MM')
ORDER BY month;
```

Step 9: Top Products by Volume

```sql
```

```sql
-- Query 5: Products by units sold
SELECT
    product,
    SUM(quantity) as total_units_sold,
    SUM(total) as revenue
FROM sales
GROUP BY product
ORDER BY total_units_sold DESC
LIMIT 5;
```

Part 3: Create a Reusable View

Step 10: Create Product Performance View

```sql
-- Create a view for product performance
CREATE VIEW product_performance AS
SELECT
    product,
    COUNT(*) as transaction_count,
    SUM(quantity) as total_units,
    SUM(total) as total_revenue,
    ROUND(AVG(total), 2) as avg_transaction_value,
    ROUND(SUM(total) * 100.0 / (SELECT SUM(total) FROM sales), 2) as revenue_percentage
FROM sales
GROUP BY product
ORDER BY total_revenue DESC;
```

Step 11: Test the View

```sql
-- Query the view
SELECT * FROM product_performance;
```

Part 4: Document Your Work

Step 12: Add Documentation

At the top of your sales_queries.sql file, add:

```sql
```

```
/*
==============================================
SALES DATABASE QUERIES
==============================================
Database: sales_db
Table: sales
Created: [Today's Date]
Author: [Your Name]

Purpose: Analysis queries for 2024 sales data

Available Objects:
- Table: sales (raw sales data)
- View: product_performance (aggregated product metrics)

Usage:
Run queries individually to answer specific business questions.
Use product_performance view for quick product analysis.
==============================================
*/
```

Then add all your queries below this header.